

checkitoff

User manual

Contents

1. What checkitoff is for	2
2. Installation and compiling	2
3. Your first checklist	2
3.1. Recommended workflow	6
4. Marking items: <code>check</code> and <code>na</code>	6
4.1. The point-marker form: <code>check(id)</code> alone	7
5. Reading the grid	8
5.1. Diagnostics	8
5.2. Strict mode	11
6. Quoting the real wording: <code>excerpt-of</code>	11
7. The page-break idiom	12
8. Styling the grid: <code>set-style</code>	13
9. Wiring a real project	16
10. Built-in checklists	17
11. Checkitoff in the contexture ecosystem	17
11.1. What contexture does	17
11.2. <code>@preview/palimpsest</code> : manuscript revisions and reviewer letters	18
11.3. Combining the two	18
11.4. Where to go next	18

1. What checkitoff is for

Reporting guidelines (CONSORT for randomised trials, PRISMA for systematic reviews, SPIRIT for trial protocols, STARD, STROBE...) ask an author to prove that a manuscript reports a fixed list of items — and, for most journals, to submit a completed grid citing the exact page where each one appears. Written by hand, that grid is done once, then quietly goes stale the first time a paragraph moves.

Checkitoff automates it. Mark each item where it actually appears in your manuscript — `#check("6a") [...]` around the sentence describing your primary outcome, say — and one compile produces both:

- `manuscript.pdf` — exactly what you're submitting, with no visible trace of any `check()` call;
- `checklist.pdf` — the official grid, filled in, citing the real page number each item landed on **in that same compile**. Move a paragraph, recompile, and the numbers are simply right again.

This only works because both documents come out of a single Typst compile that can see its own final layout. Checkitoff itself doesn't implement that part: it's built on `contexture`, a small shared package that does the actual multi-document plumbing and is installed alongside it (see the next chapter). You don't need to learn `contexture` to use checkitoff — everything you need is shown here, one step at a time. The closing chapter, "Checkitoff in the contexture ecosystem," explains what `contexture` actually does and introduces `@preview/palimpsest`, a sibling package for tracked manuscript revisions and reviewer response letters, for anyone who wants the bigger picture or needs to combine the two.

2. Installation and compiling

```
#import "@preview/checkitoff:0.1.0": *
#import "@preview/contexture:0.1.0": bundle
```

A manuscript plus its completed checklist compiles with one command:

```
typst compile --features bundle --format bundle main.typ
```

This produces `manuscript.pdf` and `checklist.pdf` from the same compile. A second, optional compile is useful while drafting:

```
typst compile --features bundle --format bundle --input preview=true main.typ
```

This produces `manuscript-preview.pdf` only (no `checklist.pdf` in this compile — see why, below) with every `check()`'d span lightly highlighted and tagged with its item id, so you can see at a glance what's covered so far and where. Nothing about it ever reaches the real submission.

Every example in this manual is a single, self-contained file, compiled directly — the simplest way to show what each function does on its own. `check()` and `na()` even work in a plain `typst compile`, with no bundle at all: that's the form used for the very next example. A real, multi-file project instead wires its manuscript and its checklist together explicitly; that pattern is covered once you've seen the pieces it's built from, in Wiring a real project.

3. Your first checklist

The smallest complete example: a four-item made-up checklist (a real project would pass `checklists.consort` or another built-in grid instead — see Built-in checklists — but a tiny one keeps the whole grid on one page here), a short manuscript, and two `check()` calls plus one `na()`.

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

// A tiny, made-up grid — not a real reporting guideline — kept small so
// checklist.pdf fits on one page for this illustration; a real project
// passes e.g. `checklists.consort` here instead (see the "Built-in
// checklists" chapter).
#let tiny = (
```

```

name: "TINY",
full-name: [Tiny reporting checklist],
items: (
  (section: "Title and abstract", topic: "Title", group: none, id: "1a",
    description: [Identification as a randomised trial.]),
  (section: "Methods", topic: "Sequence generation", group: "Randomisation", id: "10",
    description: [Method used to generate the random allocation sequence.]),
  (section: "Methods", topic: "Allocation concealment", group: "Randomisation", id: "11",
    description: [Mechanism used to conceal the allocation sequence.]),
  (section: "Discussion", topic: "Interpretation", group: none, id: "14",
    description: [Interpretation consistent with results, balancing benefits and harms.]),
),
)

#show: contexture.bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 10.5pt)
    body
  },
  documents: (checklist(checklist: tiny),),
)

= Title and abstract

#check("1a")[Effect of Drug X on Outcome Y: a randomised controlled trial.]

= Methods

#check("10")[Participants were randomly assigned via a computer-generated
sequence.]

#check("11")[Allocation was concealed using sequentially numbered, opaque,
sealed envelopes.]

= Discussion

This short illustrative manuscript omits a full interpretation section.

#na("14", reason: [Omitted from this compact illustrative example.])

```

A few things to notice in that file, top to bottom:

1. tiny is nothing but a dictionary: a name, a full-name, and a list of items, each with a section, an optional topic/group, an id, and a description. Nothing here is specific to checkitoff's internals — it's the same shape every built-in checklist uses, and the same shape a house checklist of your own would use.
2. #show: contexture.bundle.with(...) is the one line of setup a real project needs: it tells Typst which checklist is active (checklist(checklist: tiny)) and applies a page template to the manuscript. Everything after it is your manuscript, written exactly as you'd write it without checkitoff at all.
3. #check("1a")[...] wraps a passage of the manuscript and records "this is where item 1a is answered." #na("14", reason: [...]) instead declares, in one line, that item 14 doesn't apply here — with a justification, not a silent omission.

Compiled plain, manuscript.pdf shows nothing but ordinary prose — no box, no id, no visible trace of check() at all:

Title and abstract

Effect of Drug X on Outcome Y: a randomised controlled trial.

Methods

Participants were randomly assigned via a computer-generated sequence.

Allocation was concealed using sequentially numbered, opaque, sealed envelopes.

Discussion

This short illustrative manuscript omits a full interpretation section.

Compiled again with `--input preview=true` — a drafting view only, never part of the real submission — each checked span is lightly highlighted with its item id superscripted, so you can see coverage at a glance while writing (`checklist.pdf` isn't produced in this compile at all — see why below):

Title and abstract

Effect of Drug X on Outcome Y: a randomised controlled trial. ^[1a]

Methods

Participants were randomly assigned via a computer-generated sequence. ^[10]

Allocation was concealed using sequentially numbered, opaque, sealed envelopes. ^[11]

Discussion

This short illustrative manuscript omits a full interpretation section.

And here is `checklist.pdf`, from the first, plain compile — the actual deliverable:

Tiny reporting checklist

Section / Topic	No	Description	Reported on page no.
Title and abstract			
Title	1a	Identification as a randomised trial.	1
Methods			
<i>Randomisation</i>			
Sequence generation	10	Method used to generate the random allocation sequence.	1
Allocation concealment	11	Mechanism used to conceal the allocation sequence.	1
Discussion			
Interpretation	14	Interpretation consistent with results, balancing benefits and harms.	N/A

Not applicable

14 — Omitted from this compact illustrative example.

Worth noticing in that grid: “Randomisation” (item 10/11’s shared `group`) renders as its own band nested inside the “Methods” section band — an item’s `group`, when it has one, always gets this treatment. Item 14 shows N/A in the Page column, with its justification printed verbatim underneath, in the “Not applicable” block — both come directly from the single `na("14", reason: [...])` call above, with no `check()` anywhere for that id. If an

item had been left neither `check()`'d nor `na()`'d, its Page cell would instead show a small warning marker — covered in full in Reading the grid, once the individual marking functions have been introduced properly.

3.1. Recommended workflow

The example above is the whole mechanism; a real manuscript just repeats the same two calls, `check()` and `na()`, at every relevant spot. In practice:

1. Pick a checklist — one of the built-in ones (below), or your own dictionary of the same shape.
2. Write the manuscript as usual, wrapping each passage that answers an item in `check(id)[...]`, and grouping every genuinely inapplicable item under `na(id, reason: [...])`.
3. While drafting, compile with `--input preview=true` from time to time — a quick visual check of what's covered and what isn't yet, with no effect on the real manuscript.
4. Once the draft feels complete, compile normally and read `checklist.pdf`. Its Diagnostics block (below) lists anything unresolved: an item never covered, a `check()` with nothing in it, an id that doesn't match any item, or a genuine contradiction.
5. Fix each one — add the missing `check()`, or an `na()` with a real reason — and recompile.
6. Before submission, compile once with `strict: true` (below): every remaining diagnostic becomes a hard compile error instead of a soft marker, a clean pass-fail gate.
7. Submit `manuscript.pdf` and `checklist.pdf` together — they came from the same compile, so the page numbers in one are guaranteed to match the other.

4. Marking items: `check` and `na`

`check(id, body)` anchors `body` to item `id` of whichever checklist is active for this compile, and renders `body` exactly as written. Zero visual footprint in the plain compile — safe inside any journal template's flow. Under `--input preview=true`, `body` gets a light highlight with the item id superscripted, purely as a drafting aid.

`na(id, reason: none)` declares item `id` not applicable to this manuscript, with an optional justification. Renders nothing at the call site — it's typically grouped in one dedicated block rather than scattered through the text, as in the example above.

A minimal, standalone example — no bundle involved, just `check()` in a plain file:

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

The primary outcome was #check("6a")[change in disease activity score from
baseline to week 12], assessed by a rater blinded to group assignment.
```

Compiled once, plain:

The primary outcome was change in disease activity score from baseline to week 12, assessed by a rater blinded to group assignment.

Once more with `--input preview=true`:

The primary outcome was change in disease activity score from baseline to week 12^[6a], assessed by a rater blinded to group assignment.

`check()` doesn't need to know which checklist is active, or even that one exists — it just anchors `body` under `id`; whether that `id` means anything is only decided later, when the grid is built. That's why this file compiles and renders correctly entirely on its own, with no `#show: bundle.with(...)` anywhere. It's also why an id that turns out not to match any item is a diagnostic caught later, in the grid, rather than an error here.

An item can legitimately be checked more than once — a method described once and its rationale discussed elsewhere, say — in which case the grid lists every page it was found on. The one deliberate exception is a single passage that straddles a page break: see The page-break idiom for the recommended way to get a correct, two-page citation for that case specifically.

`na()` is the explicit counterpart to `check()`: some items genuinely don't apply to a given manuscript (a trial with no serious adverse events has nothing to say for a "Harms" item, for instance). Declaring it, with a reason, turns "item never mentioned" from an oversight worth flagging into a documented, deliberate choice.

4.1. The point-marker form: `check(id)` alone

`check(id, body)` renders `body` — and that's the right call almost every time, since most items simply live at their own spot in the manuscript. Occasionally, though, the passage an item covers is already being rendered by something else — most commonly another package's own marking call, when what a reviewer asked you to change happens to **be** your answer to a checklist item. Calling `check(id, body)` there too would print the same text a second time, since it always renders `body` itself.

For exactly that case, `check(id)` — one argument, no `body` — only ever registers the item's location; it never renders anything, ever. `body` must already be displayed by something else.

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

#let tiny = (
  name: "TINY",
  full-name: [Tiny reporting checklist],
  items: (
    (section: "Methods", topic: "Outcome assessment", group: none, id: "6a",
      description: [How the primary outcome was assessed.]),
  ),
)

#show: contexture.bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 10.5pt)
    body
  },
  documents: (checklist(checklist: tiny)),
)

// check("6a") with no body registers item "6a" here without rendering
// anything at all --- the sentence itself is printed directly, right
// below, exactly once. Compare with marks-check-basics.typ: same
// manuscript wording, but check(id, body) both anchors AND renders
// (with a highlight box under preview=true); check(id) only anchors.
The primary outcome was change in disease activity score from baseline
to week 12, assessed by a rater blinded to group assignment. #check("6a")
```

Compiled once, plain — the sentence appears exactly once, exactly as written:

The primary outcome was change in disease activity score from baseline to week 12, assessed by a rater blinded to group assignment.

Once more with `--input preview=true` — a small superscripted id, but no highlight box: there's no `body` here to wrap one around, but the id is still shown as a drafting aid:

The primary outcome was change in disease activity score from baseline to week 12, assessed by a rater blinded to group assignment. ^[6a]

The item still resolves correctly in `checklist.pdf`, exactly as if it had been written `check(id, body)`. The only function that notices the difference is `excerpt-of` (below): with no `body` to quote, an occurrence marked this

way simply contributes no excerpt, which is the honest answer, not an error. The “blank content” diagnostic never fires on it either — a missing body here is the deliberate shape of this form, not the mistake it would be for `check(id)[]`.

The most common reason to reach for this form — combining `checkitoff` with a package that already renders the text, such as `@preview/palimpsest`’s tracked-changes marks — is covered in full in the closing chapter, `Checkitoff` in the `contexture` ecosystem.

5. Reading the grid

`render-checklist(checklist:, title: auto)` is the function that draws the grid you already saw in *Your first checklist*: every official item, grouped by section (and, when the checklist has one, by a mid-level group inside a section), each item’s resolved page number(s), and a Diagnostics block listing anything that doesn’t add up. In an ordinary project you never call it directly — the `checklist(...)` satellite (below) calls it for you — but it’s exported on its own too, in case a project ever needs the grid outside the usual two-document setup.

A checklist is plain data, as already seen: `name`, `full-name`, `items`, plus optional `headers`, `style`, and `citation` fields covered later in this manual. `checklist:` never defaults to anything, even though `CONSORT` ships built in — an explicit choice costs nothing and avoids a silent, surprising default once a project is juggling more than one grid.

5.1. Diagnostics

Five situations are flagged, always in the same way, so `strict: mode` (below) catches every one of them consistently:

- an item never `check()`’d or `na()`’d at all — **not covered**;
- a `check()` call whose content is blank — most often a copy-paste slip, where the id was moved but the text wasn’t filled in;
- an id used by `check()/na()` that matches no item in the active checklist — typically a typo, or a leftover from switching checklists;
- the same id both `check()`’d and `na()`’d — a real contradiction: an item can’t be simultaneously reported somewhere and declared not applicable;
- the same id `na()`’d more than once.

The first three (not covered / blank / unknown id) show up right in the Page column, as a  marker; every diagnostic’s full sentence, including the two that have no single cell of their own (unknown id, duplicate `na()`), is listed underneath the table, in a dedicated Diagnostics block.

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

// Isolates every diagnostic case grid.typ can raise, one per line, against
// a tiny made-up checklist so each is trivially visible in checklist.pdf.

#let tiny = (
  name: "TINY-TEST",
  full-name: [Tiny test checklist (not a real reporting guideline)],
  items: (
    (section: "Section A", topic: "Topic 1", group: none, id: "t1",
      description: [Covered normally --- the ordinary, non-diagnostic case.]),
    (section: "Section A", topic: "Topic 1", group: none, id: "t2",
      description: [Never check()'d or na()'d --- demonstrates "not covered".]),
    (section: "Section A", topic: "Topic 2", group: none, id: "t3",
      description: [check()'d with blank content --- demonstrates "blank content".]),
    (section: "Section B", topic: "Topic 3", group: none, id: "t4",
      description: [Both check()'d and na()'d --- demonstrates the conflict diagnostic.]),
    (section: "Section B", topic: "Topic 3", group: none, id: "t5",
      description: [na()'d twice --- demonstrates "declared N times".]),
    (section: "Section B", topic: "Topic 4", group: "Sub-group demo", id: "t6",
      description: [Under a group header, first of a 2-row Topic rowspan.]),
    (section: "Section B", topic: "Topic 4", group: "Sub-group demo", id: "t7",
      description: [Under the same group header, second row of the same Topic rowspan.]),
  ),
)
```

```

#show: contexture.bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 9.5pt)
    body
  },
  documents: (checklist(checklist: tiny),),
)

t1 --- the normal case, no diagnostic expected:

#check("t1")[Normal content, correctly covered.]

t2 --- never covered.

t3 --- blank content:

#check("t3")[]

t4 --- both check() and na() on the same id:

#check("t4")[Covered anyway.]
#na("t4", reason: [Declared not applicable here only to test the conflict
  with check().])

t5 --- na() declared twice for the same id:

#na("t5", reason: [First declaration.])
#na("t5", reason: [Second declaration --- deliberate duplicate.])

t6/t7 --- sharing the "Sub-group demo" group and the same Topic:

#check("t6")[First item of the group demonstration.]
#check("t7")[Second item of the same group, same topic.]

An id entirely unknown to the "tiny" checklist, referenced once via
`check()` and once via `na()`:

#check("zzz")[This id doesn't exist in any grid --- unknown id via check().]
#na("yy", reason: [Neither does this one --- unknown id via na().])

```

Tiny test checklist (not a real reporting guideline)

Section / Topic	No	Description	Reported on page no.
Section A			
Topic 1	t1	Covered normally — the ordinary, non-diagnostic case.	1
	t2	Never check()’d or na()’d — demonstrates “not covered”.	⚠
Topic 2	t3	check()’d with blank content — demonstrates “blank content”.	1
Section B			
Topic 3	t4	Both check()’d and na()’d — demonstrates the conflict diagnostic.	⚠ N/A
	t5	na()’d twice — demonstrates “declared N times”.	N/A
Sub-group demo			
Topic 4	t6	Under a group header, first of a 2-row Topic rowspan.	1
	t7	Under the same group header, second row of the same Topic rowspan.	1

Diagnostics

- ⚠ item t2: not covered (no check(), no na())
- ⚠ item t4: marked not applicable (na) but also checked by check() (p. 1)
- ⚠ check("t3"): blank content (manuscript p. 1)
- ⚠ check("zzz"): unknown id for TINY-TEST (manuscript p. 1)
- ⚠ na("yyy"): unknown id for TINY-TEST (declared p. 1)
- ⚠ na("t5") declared 2 times

Not applicable

- t4 — Declared not applicable here only to test the conflict with check().
- t5 — First declaration.

This same example also shows what happens when consecutive items share one topic (t6/t7, both “Topic 4”, under the “Sub-group demo” group): the Topic cell spans both rows instead of repeating.

5.2. Strict mode

By default, every diagnostic above is a soft, visible marker — easy to spot while drafting, but it won't fail a build on its own. `strict: true`, passed to `contexture.bundle(...)` (not to `checklist(...)` — strictness is a property of the whole compile, so it covers any other `contexture`-based package sharing it too), turns every one of them into a hard compile error instead:

```
#show: contexture.bundle.with(
  strict: true,
  documents: (checklist(checklist: my-checklist)),
)

= Manuscript

// No check("s1") anywhere below --- this manuscript now
// FAILS TO COMPILE under strict: true, with a real compile
// error naming the uncovered item, instead of a soft marker
// sitting quietly in checklist.pdf.
```

The positive case — everything covered, `strict: true` compiles cleanly, with no diagnostic anywhere:

```
#show: contexture.bundle.with(
  strict: true,
  documents: (checklist(checklist: my-checklist)),
)

= Manuscript

#check("s1")[This item is covered here.]
#na("s2", reason: [Not applicable to this manuscript, with a
  proper justification.])
```

A real project typically reserves `strict: true` for a CI compile or a final pre-submission check — a hard gate — while drafting locally without it, so an incomplete manuscript still produces a readable `checklist.pdf` with markers instead of refusing to build at all.

6. Quoting the real wording: excerpt-of

`excerpt-of(id, quotes: false, show-page: false, on-empty: none)` re-emits the exact content `check()` anchored to `id` — one block per occurrence, so an item checked in two places yields two excerpts. It isn't part of the official grid, which only ever has a page-number column; it's for a supplementary appendix some journals or protocols additionally want, with the exact wording quoted next to each item, or for an internal compliance review.

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

// Demonstrates excerpt-of(): re-emitting the real, verbatim content
// anchored by check(), e.g. for a supplementary "compliance appendix"
// listing exact wording next to each item --- not part of the official
// CONSORT grid (which only has a page-number column), so this lives in
// the manuscript itself, not in checklist.pdf.

#show: contexture.bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 10.5pt)
    body
  },
  documents: (checklist(checklist: checklists.consort)),
)

= Methods

#check("17a")[The allocation sequence was generated by an independent
  statistician using a computer-based random-number generator.]

#check("18")[Allocation was concealed using sequentially numbered, opaque,
  sealed envelopes.]
```

= Appendix: verbatim compliance excerpts

```
*Item 17a* (Sequence generation): #excerpt-of("17a", quotes: true, show-page: true)

*Item 18* (Allocation concealment mechanism): #excerpt-of("18", quotes: true, show-page: true)

*Item 19* (Implementation) --- never checked in this manuscript, default
`on-empty: none` renders nothing between the colon and the period: #excerpt-of("19").

*Item 20a* (Blinding) --- never checked either, but with an explicit
fallback: #excerpt-of("20a", on-empty: [_ (not reported)]).
```

Methods

The allocation sequence was generated by an independent statistician using a computer-based random-number generator.

Allocation was concealed using sequentially numbered, opaque, sealed envelopes.

Appendix: verbatim compliance excerpts

Item 17a (Sequence generation): **p. 1** — “The allocation sequence was generated by an independent statistician using a computer-based random-number generator.”

Item 18 (Allocation concealment mechanism): **p. 1** — “Allocation was concealed using sequentially numbered, opaque, sealed envelopes.”

Item 19 (Implementation) — never checked in this manuscript, default on-empty: none renders nothing between the colon and the period: .

Item 20a (Blinding) — never checked either, but with an explicit fallback: *(not reported)*.

quotes: true wraps a textual excerpt in real quotation marks, and silently declines on anything that isn't text — a figure, a table, a block equation. on-empty (nothing, by default) is deliberately **not** a diagnostic the way an uncovered item is in the grid: this function can be called from inside the manuscript itself, where nothing should ever render a warning box in the real, submitted deliverable — an uncovered item is already flagged exactly once, safely, in `checklist.pdf`.

7. The page-break idiom

Checkitoff doesn't try to detect a passage that straddles a page break automatically — there's no reliable way to tell “one logical passage split across pages” apart from “two genuinely separate mentions of the same item.” The recommended idiom instead: call `check()` a second time, with the **same** id, right after the break.

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

// The recommended idiom for an item whose content straddles a page break
// (no automatic span detection): a check() call at the start of the
// passage, and a second check() call with the *same* id right after the
// page break. pages-of's dedup means this naturally reports both pages,
// not one merged range and not just the first page.

#show: contexture.bundle.with(
  template: body => {
    set page(paper: "a4", margin: 2.5cm)
    set text(size: 11pt)
    body
  },
  documents: (checklist(checklist: checklists.consort)),
)

= Methods

#check("13") [
```

```

The intervention consisted of a daily 10 mg dose of Drug X, administered
orally with food, for a total of 12 weeks. The comparator was a
matching placebo tablet, identical in appearance, taste and packaging,
administered on the same schedule.
]

#lorem(200)
#pagebreak()

#check("13")[
  As introduced above, the comparator (placebo) was manufactured by the
  same facility as Drug X to guarantee identical appearance, and both
  were dispensed in blister packs labelled only with the participant's
  allocation number.
]

= Results

#lorem(30)

```

The grid reports both pages, not a merged range and not just the first:

Section / Topic	No	Description	Reported on page no.
Changes to trial protocol	10	Important changes to the trial after it commenced including any outcomes or analyses that were not prespecified, with reason	▲
Trial setting	11	Settings (e.g., community, hospital) and locations (e.g., countries, sites) where the trial was conducted	▲
Eligibility criteria	12a	Eligibility criteria for participants	▲
	12b	If applicable, eligibility criteria for sites and for individuals delivering the interventions (e.g., surgeons, physiotherapists)	▲
Intervention and comparator	13	Intervention and comparator with sufficient details to allow replication. If relevant, where additional materials describing the intervention and comparator (e.g., intervention manual) can be accessed	1, 2
Outcomes	14	Pre-specified primary and secondary outcomes, including the specific measurement variable (e.g., systolic blood pressure), analysis metric (e.g., change from baseline, final value, time to event), method of aggregation (e.g., median, proportion), and time point for each outcome	▲
Harms	15	How harms were defined and assessed (e.g., systematically, non-systematically)	▲
Sample size	16a	How sample size was determined, including all assumptions supporting the sample size calculation	▲
	16b	Explanation of any interim analyses and stopping guidelines	▲
<i>Randomisation</i>			
Sequence generation	17a	Who generated the random allocation sequence and the method used	▲
	17b	Type of randomisation and details of any restriction (e.g., stratification, blocking and block size)	▲

Two genuinely separate mentions of the same item, landing on the same page, collapse to one page number instead — the same rule, the other direction.

8. Styling the grid: `set-style`

Every visual knob the grid and `check()`'s preview highlighting use goes through one shared mechanism, resolved in three layers, each overriding only what the previous layer left unset:

1. `checkitoff`'s own package default — generic and checklist-agnostic (portrait A4, no color, “use whatever the ambient template already set”);
2. the active checklist's own `style:` field, if it has one — CONSORT's real landscape A4 layout and column widths, taken from its official source document, is exactly this layer;
3. whatever you explicitly ask for via `set-style(...)`.

`set-style(...)` takes every knob as a keyword, `auto` by default (“don’t touch this one”); repeated calls merge rather than reset:

```
#set-style(
  preview-color: auto,          // check()'s preview-mode highlight color
  show-id: auto,                // superscript the id in preview mode?
  paper: auto,                  // grid page size, e.g. "a4", "us-letter"
  landscape: auto,              // grid page orientation
  columns: auto,                // 4 column widths, e.g. (18%, 7%, 1fr, 10%)
  font: auto,                   // grid text font
  text-size: auto,              // grid text size
  header-fill: auto,            // header row background
  header-text-color: auto,
  section-fill: auto,           // section band background
  section-text-color: auto,
  group-fill: auto,             // mid-level group band background
  group-text-color: auto,
)
```

An override on a checklist with no `style:` of its own:

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

// "tiny" has no style: field of its own, so it renders with checkitoff's
// package-default-style until set-style() below overrides it.
#let tiny = (
  name: "TINY",
  full-name: [Tiny reporting checklist],
  items: (
    (section: "Methods", topic: "Outcome assessment", group: none, id: "1",
      description: [How the primary outcome was assessed.]),
    (section: "Methods", topic: "Randomisation", group: none, id: "2",
      description: [How the allocation sequence was generated.]),
  ),
)

#set-style(
  columns: (22%, 6%, 1fr, 12%),
  header-fill: rgb("#2e3436"),
  header-text-color: white,
  section-fill: rgb("#eeeeec"),
)

#show: contexture.bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    body
  },
  documents: (checklist(checklist: tiny),),
)

#check("1")[The primary outcome was assessed by a rater blinded to group
assignment.]

#check("2")[Participants were randomly assigned via a computer-generated
sequence.]
```

Tiny reporting checklist

Section / Topic	No	Description	Reported on page no.
Methods			
Outcome assessment	1	How the primary outcome was assessed.	1
Randomisation	2	How the allocation sequence was generated.	1

`preview-color/show-id` also apply to `check()`'s own preview-mode highlight, independently of any checklist — `check()` doesn't know which checklist is active, by design:

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

#set-style(preview-color: rgb("#c01c28"), show-id: false)

The primary outcome was #check("6a")[change in disease activity score from
baseline to week 12], assessed by a rater blinded to group assignment.
```

The primary outcome was `change in disease activity score from baseline to week 12`, assessed by a rater blinded to group assignment.

These values always win, over both checkitoff's package default and the active checklist's own `style`: — your explicit ask, for the manuscript you're compiling right now, is the most specific signal available. A checklist that ships its own faithful `style`: (CONSORT, PRISMA, ...) keeps looking like its real source document by default; `set-style(...)` is for the cases where that's not what you want — matching a specific journal's house style, or simply personal preference.

9. Wiring a real project

Every example so far is one file, compiled directly. A real project instead usually keeps the manuscript's prose in its own file and wires everything together from a small `main.typ`:

```
#show: contexture.bundle.with(
  template: my-manuscript-template,
  documents: (
    checklist(checklist: checklists.consort),
    // ... any other contexture-based package's satellite here too
  ),
)

#include "manuscript.typ"
```

`checklist(checklist:, grid-template: auto)` is what describes the checklist for `contexture.bundle` to build. `checklist:` has no default, for the same reason as `render-checklist`'s own `checklist:` above. `grid-template:` (`auto` = leave it alone) is applied to the grid separately from the manuscript's own `template:` — most journals want CONSORT's own official table layout on the checklist page, not the manuscript's own house style.

Restricting a compile to fewer documents. `--input only=<comma-separated names>` restricts a single compile to just the manuscript plus the named documents — `--input only=` with nothing after it produces the manuscript alone, with no checklist at all. Handy for a fast preview while drafting a long manuscript, where rebuilding the grid every time is unwanted overhead.

Why doesn't `--input preview=true` produce a `checklist.pdf`? `check()`'s preview highlighting adds a small box around each checked span, which can shift where a page breaks. A grid built from that layout could then report page numbers that don't match the manuscript actually being submitted — worse than not producing one at all. `checklist.pdf` is only ever built from the one, real, plain compile.

Does the checklist compile when the manuscript doesn't? No. The manuscript and the checklist are two documents produced by the very same Typst compile, so a hard error anywhere in the manuscript aborts the whole compile, `checklist.pdf` included. A `check()/na()` diagnostic never does this on its own — that's exactly what non-strict mode is for — only a genuine error in the manuscript itself (an undefined function, a malformed table, ...) does.

10. Built-in checklists

<code>checklists.</code> key	Guideline	Items	Notes
<code>consort</code>	CONSORT 2025 (randomised trials)	42	Landscape A4; one mid-level group, “Randomisation” (17a–21d).
<code>prisma</code>	PRISMA 2020 (systematic reviews)	42	Landscape US Letter; no mid-level groups.
<code>spirit</code>	SPIRIT 2025 (trial protocols)	53	Landscape US Letter.
<code>stard</code>	STARD 2015 (diagnostic accuracy studies)	34	Portrait A4.
<code>strobe.cohort</code>	STROBE (cohort studies)	22	Portrait A4.
<code>strobe.case_control</code>	STROBE (case-control studies)	22	Portrait A4.
<code>strobe.cross_sectional</code>	STROBE (cross-sectional studies)	22	Portrait A4.

Every entry is transcribed from its official source document, including its citation and license notice — reproduced verbatim in the “citation” block at the bottom of `checklist.pdf` — and its real column widths and section colors, read directly from the source file rather than guessed. `strobe` is itself a dictionary of the three study-design variants above rather than one single checklist: there is no fourth, “combined” variant, since that source bundles all three designs’ wording into a single item per row, which doesn’t fit the one-description-per-id shape every other checklist here uses.

A checklist is plain data, as the quickstart already showed — nothing about `check()`, `na()`, `render-checklist`, or the `checklist(...)` satellite is specific to CONSORT or to any built-in grid above. A project with its own house checklist, or an emerging reporting guideline not built in yet, simply passes its own dictionary of the same shape in `checklist`: instead.

11. Checkitoff in the contexture ecosystem

Checkitoff is one of two packages built on `contexture`, a small shared package neither of them ships duplicated logic for. This chapter explains what `contexture` actually contributes, introduces the other package built on it, and covers what changes when both are used together.

11.1. What `contexture` does

Everything in this manual that looks up a **real** page number across two documents — `checklist.pdf` citing exactly where in `manuscript.pdf` each item landed — relies on Typst’s experimental bundle export, which lets one compile produce several documents that can query each other’s final layout. `contexture` is the small toolkit that turns that raw capability into something a package author can build on without reinventing it each time:

- an **anchor** primitive — mark a spot in one document, read it back from any other, by its real page;
- a **shared compile pilot** (`bundle`) — the single point that ever calls Typst’s own `document(...)`, so that checkitoff’s grid and, say, another package’s own generated document can both be listed side by side without competing to own the compile;
- two independent **compile flags**, `variant` and `preview` — `preview` is what powers `--input preview=true` throughout this manual; `variant` is a second, independent axis a package can use for its own purposes (palimpsest uses it for clean vs. tracked-changes output, below);
- a shared **diagnostics** mechanism and `strict` flag — what every warning marker and `strict: true` in this manual are actually built from.

`checkitoff.check()` is a thin wrapper around `contexture`’s anchor primitive; `checklist(...)` is a thin wrapper around its shared compile pilot. None of this needs to be learned to use checkitoff as documented above — it’s mentioned here because the same foundation is shared with the package below, which is what makes combining the two straightforward rather than a rewrite.

11.2. @preview/palimpsest: manuscript revisions and reviewer letters

palimpsest is a sibling package for a different problem: tracking changes made to a manuscript during peer review (add, del, rep, anchored to a specific reviewer comment), and generating the tracked-changes manuscript and a reviewer response letter that cites the manuscript’s real pages — down to quoting the exact revised wording next to each response, if wanted. See @preview/palimpsest’s own manual for the full picture; nothing in it is needed to use checkitoff on its own.

11.3. Combining the two

Checkitoff’s `checklist(...)` and palimpsest’s `letter(...)` are both just descriptions of a document to build, in the same sense as `checklist(...)` was introduced in Wiring a real project — listing both under the same documents: produces, from one compile, a manuscript, its tracked-changes companion, a reviewer response letter, and a completed reporting-guideline grid, all citing each other’s real page numbers:

```
#show: contexture.bundle.with(  
  documents: (  
    palimpsest.letter(exchanges: my-exchanges),  
    checkitoff.checklist(checklist: checklists.consort),  
  ),  
)  
  
#include "manuscript.typ"
```

Two rules matter once both packages might touch the same span of text:

1. **Never nest** `check(...)` and `passage(...)` **inside each other’s body, in either direction.** Each wraps its own rendering in a way the other’s structural scan can’t see through, so nesting either way produces a false diagnostic.
2. **Don’t call** `check(...)` and `passage(...)` **as two independent, rendering siblings on the exact same wording, either.** Nothing stops you, and nothing diagnoses it — but both functions render their own body, so the same text prints twice, plainly duplicated in `manuscript.pdf`. This only bites when a reviewer’s requested change genuinely **is** the manuscript’s answer to a checklist item; two unrelated spans, the common case, have nothing to duplicate.

For that second case, use the bare `check(id)` form instead, introduced earlier: `passage(...)` stays the one call that renders the text and carries its tracked-changes marks, `check(id)` only registers the item’s coverage, with nothing left to duplicate or nest.

@preview/contexture’s own manual walks through this exact combination end to end, including compiling the tracked-and-preview manuscript together, in its chapter “Composing independent packages” — the package-agnostic version of the two rules above. @preview/palimpsest’s manual covers its own side of it, under “Combining with another contexture package.”

11.4. Where to go next

- The mechanics behind all of this, on their own, with no notion of checklists or revisions attached: @preview/contexture’s manual.
- Tracked manuscript revisions and reviewer response letters that cite the real pages: @preview/palimpsest’s manual.
- Everything about reporting-guideline checklists on their own: the rest of this manual, from Your first checklist onward.